COMP 100 - Problem Set 3

Overview

For this assignment, we'll analyze COVID-19 data from the European Centre for Disease Prevention and Control, focusing on 30 countries. The dataset includes information on populations and age demographics, as well as daily/weekly infection rates and deaths, segmented by age groups.

We'll be focusing on practicing file input/output operations, parsing, data sorting, working with lists and dictionaries. In the end, we will be able to calculate (per country):

- Cases/deaths per 1M,
- Exposure per age group, and the most/least infected age groups,
- Probability of belonging a new case to an age group,
- The distribution of deaths according to age groups

Finally, we will report extreme cases according to these statistics.

Data Source

You can access the COVID-19 data using this link. We will be working on these two datasets:

1. Daily COVID-19 Cases and Deaths (covid_data.json)

Contains information on newly reported COVID-19 cases and deaths by EU/EEA countries for specific days. [More info here]


```
{
  "records": [
    {
      "dateRep": "15/05/2021",
      "day": "15",
      "month": "05",
      "year": "2021",
      "cases": 721,
      "deaths": 14,
      "countriesAndTerritories": "Austria",
      "geoId": "AT",
      "countryterritoryCode": "AUT",
      "popData2020": "8901064",
      "continentExp": "Europe"
```

```
    },
    ...
  ]
}
```

2. 14-day Age-specific Notification Rate of New COVID-19 Cases (age_cases.json)

Includes the 14-day notification rate of newly reported COVID-19 cases per 100,000 population by age group, week, and country. [More info here].

```
[
    {
      "country": "Austria",
      "country_code": "AT",
      "year_week": "2021-19",
      "age_group": "<15yr",
      "new_cases": 156,
      "population": 1283060,
      "rate_14_day_per_100k": 16.7,
      "source": "TESSy COVID-19, national weekly data"
    },
    ...
]
```

There are lots of options for the file format such as XLSX, CSV, JSON and XML. We are going to use JSON format, which stores the data as a structured format in text-based and human-readable files.

Q1) Processing Daily Cases (15 pts)

Target File: read_data.py & Target Function: process_daily_data

Your first task is to process the daily COVID-19 cases and deaths data from the **covid_data.json** file. This involves the following steps:

Read the Data

Use the `with open` statement to load the data from **covid_data.json**. Here's how you can do it:

```
import json

with open('covid_data.json', 'r') as file:
```

```
data = json.load(file)
```

This code snippet opens the **covid_data.json** file in read mode and uses the `json.load()` function to parse the JSON file into a Python dictionary or list. You'll be working with a list of records, where each record is a dictionary containing daily COVID-19 data for a specific country.

Iterate Over Records

For each record in the list, you'll need to:

- Check if the country represented by the record is already in the db dictionary. If not, use create_country_entry to add it.
- For records within the date range from March 7, 2021, to May 3, 2021 (both are not included), use add_daily_record to append the data to the country's daily_records list.
- Use geoId as keys for db.

Dates

Use Python's `datetime` library to handle the dates in the data. You'll need to convert the dateRep string from each record to a `datetime` object to work with it. Here's an example of how to do this:

```
from datetime import datetime

date_string = record['dateRep']
date_object = datetime.strptime(date_string, '%d/%m/%Y')
```

Implement these steps in the process_daily_data function, which takes the db dictionary as its parameter and updates it in place with the processed data.

Hint

For parsing dates and performing date calculations, familiarize yourself with the datetime library by reading the Python 3 datetime documentation.

Q2) Processing Weekly Cases (15 pts)

Target File: read_data.py &Target Function: process_weekly_data

Next, you'll process the weekly COVID-19 cases data by age group from the **age_cases.json** file:

Load and Parse Data

Similar to the daily cases, open and parse the **age_cases.json** file to get a list of records, each containing weekly cases data by age group for different countries.

Update Database

For each record that falls within weeks between 9 and 18 (both are not included) of 2021:

- Ensure the country is present in the db dictionary.
- Update the population for each age group if it hasn't been set.
- Use add_weekly_case_data to append the weekly data to the country's weekly_records by age group.
- Use country_code for keys in db.

These operations should be implemented in the process_weekly_data function. This function also takes the db dictionary as its parameter and updates it with the weekly data.

Q3) Total Cases and Total Deaths (10 pts)

Target File: covid_data_analysis.py

In order to understand how bad the pandemic is for each country, we need to implement these functions first:

- get_total_cases(): Calculates the number of total cases that are stored in daily_records.
- get_total_deaths(): Calculates the number of total deaths that are stored in daily_records.
- total_cases_and_deaths_per_1m(): Scales the number of total deaths and cases to 1M population. You can use get_total_cases and get_total_deaths functions as a shortcut.

With these functions, we will be able to compare each country based on 1M population.

Q4) Age Group Analysis (10 pts)

Target File: covid_data_analysis.py

At this step, our goal is to implement functions to analyze age groups. First, we are going to calculate exposure (infection rate) per age group in calc_exposure_per_age_group() function and return the rates as a dictionary.
To calculate the infection rate, all you need to do is to divide the number of total cases of an age group by the population of the age group. An example output:

```
{
  '<15yr': 0.017913464038181382,
  '15-24yr': 0.04201250553342187,
```

```
    '25-49yr': 0.03855293226103967,
    '50-64yr': 0.03147054552624966,
    '65-79yr': 0.011292551441701803,
    '80+yr': 0.005146315722740376
}
```

which shows for example, 1.79 percent of <15yr old citizens are tested positive for COVID-19. Therefore, this function will give us information about which age group is affected the most and the least. So, we can extend our functionality to find the most and the least exposed age group. Implement the functions find_most_exposed_age_group() and find_least_exposed_age_group() and return the age group as a string.

Q5) Distributing The Number of Deaths to Age Groups (15 pts)

Target File: covid_data_analysis.py

For this task, we will make estimations about the number of deaths per age group using the weekly data, which does not directly provide death statistics by age. You'll first calculate the probability of new cases belonging to each age group, then use predefined weights to estimate the distribution of deaths.

1. Calculate Case Probabilities

Implement the calc_prob_of_cases_per_age_group() function, which calculates the ratio of the number of cases in each age group to the total number of cases. This should return a dictionary structured as follows:

```
{
    '<15yr': 0.1222859590983134,
    '15-24yr': 0.28679765802568175,
    '25-49yr': 0.26318093962981876,
    '50-64yr': 0.21483314644347676,
    '65-79yr': 0.07708841130739452,
    '80+yr': 0.035131237187663845
}
```

2. Weighted Death Probabilities

Use the given weights that represent the normalized risk of death for each age group

```
death_probs = {
```

```
    '<15yr': 0.05,
    '15-24yr': 0.05,
    '25-49yr': 0.1,
    '50-64yr': 0.2,
    '65-79yr': 0.25,
    '80+yr': 0.35
}
```

3. Distribute Deaths

In the distribute_deaths_to_age_groups() function:

- Scale the calculated case probabilities by the death probabilities.
- Normalize these scaled values so their sum equals 1.
- Retrieve the total number of deaths using get_total_deaths().
- Multiply the normalized, weighted probabilities by the total number of deaths to estimate the distribution of deaths across the age groups.

The function should return both the total number of deaths and the estimated distribution of these deaths across the age groups as a dictionary.

Hint

Normalization ensures that the weighted probabilities form a valid distribution (i.e., their sum is exactly 1), allowing the deaths to be proportionally allocated among age groups.

Q6) Sorting: Finding Top-K Cases and Top-K Deaths (20 pts)

Target File: covid_data_analysis.py

In this part, our goal is to find the dates where the number of deaths and the number of cases is the highest. This approach will help us to understand what are the peak dates for a specific country. In this section, we provide some functionality for you. Analyze the functions find_top_k_cases and find_top_k_deaths. They both make use of sort_records_by function to find the sorted version of the cases and deaths and return them after with a string converter function daily_records_to_string. The only remaining thing is some sorting mechanism which can sort a list of numbers and returns the indices according to the original list that would give the sorted list.
Implement the function `sort_records_by(country_obj, param)`. Here, param determines what type of record (cases or deaths) that are going to be sorted. You can implement any sorting algorithm you want. You can use the `.sort()` method of lists.

Q7) Processing Big Data (15 pts) Target File: covid_impact.py

Part A: Analyzing COVID-19 Impact

Implement the analyze_covid_impact(db) function in covid_impact.py file to determine:

- The most and least exposed age groups in the EU.
- The country with the highest deaths per 1M population, along with the number.
- The country with the highest cases per 1M population, along with the number.

Use functions such as total_cases_and_deaths_per_1m and calc_exposure_per_age_group for calculations.

Part B: Writing to a JSON File

After completing the data analysis in Part A, write the results to a JSON file named **covid_impact_summary.json**. This file should include a summary of the top cases, top deaths, and exposure rates.
Below is an example of what the output file **covid_impact_summary.json** might look like:

```
{
    "Highest_Deaths_per_1M": {
        "Country": "Italy",
        "Deaths_per_1M": 1523
    },
    "Highest_Cases_per_1M": {
        "Country": "Czech Republic",
        "Cases_per_1M": 15827
    },
    "Most_Exposed_Age_Group": "50-64yr",
    "Least_Exposed_Age_Group": "<15yr"
}
```

This file provides a structured summary of the COVID-19 impact, including the countries with the highest death and case rates per million population, and identifies the most and least exposed age groups based on the analysis performed.

Use this code to write in a json file.

```
with open('covid_impact_summary.json', 'w') as file:
      json.dump(summary, file, indent=4)
```